

PowerVR 與 Arm Mali GPU Shader 架構深層剖析、硬體分數板實作機制、SystemC 微架構模擬與效能計數器指南

0. 執行摘要與架構設計哲學概論 (Executive Summary & Architectural Philosophy)

行動端與嵌入式圖形處理器 (Mobile & Embedded GPU) 的設計受限於嚴苛的熱設計功耗 (TDP, 通常在 2W 至 5W 之間) 以及受限的 LPDDR 記憶體頻寬。在這一物理限制下，Imagination Technologies 的 PowerVR 系列與 Arm 的 Mali 系列開創了兩條截然不同卻又殊途同歸的微架構演進路線：

1. PowerVR 的純粹延遲著色哲學 (Pure TBDR + HSR)：

PowerVR 自始至終將「絕對不為最終不可見的像素執行片段著色器 (Fragment Shader)」視為最高準則。其核心專利 HSR (Hidden Surface Removal, 隱藏面剔除) 配合獨立的 ISP (Image Synthesis Processor) 硬體，在圖元完全光柵化後、進入著色運算前，透過硬體射線投射與深度排序完全解析不透明圖元的空間覆蓋關係。只有勝出的可見幾何才會被派發至 USC (Unified Shading Cluster) 進行計算。

2. Arm Mali 的平鋪實用主義演進 (TBR with FPK & Valhall SIMT)：

Mali 在架構發展上經歷了數次重大範式轉移：從早期基於 VLIW/SIMD 向量管線的 Midgard，到採用指令子句 (Clause-based) 與四執行緒微向量束的 Bifrost，再到現代全面轉向 16 執行緒純標量 SIMT 執行模型的 Valhall 與第 5 代架構 (5th Gen)。Mali 採用傳統 TBR (Tile-Based Rendering)，輔以 Early-Z 與創新的 FPK (Forward Pixel Kill, 前向像素終止) 技術，在 FIFO 佇列中動態剔除被覆蓋的像素，同時以更具彈性的純標量 16-wide Warp 與現代化硬體分數板最大化 ALU 運算密度與編譯器最佳化空間。

本文件基於公開晶片架構白皮書、開放原始碼驅動 (Mesa Panfrost 與 Mesa PvrGPU)、專利文獻以及本專案驗證之 SystemC 36 模組微架構模擬器，巨細靡遺剖析兩者在 Shader Core、硬體分數板 (Hardware Scoreboard)、SystemC 模擬機制與公開效能計數器上的技術實作。

1. PowerVR vs. Arm Mali GPU Shader 設計的異同深度比較

下表總結了 PowerVR (以 Rogue / Furian / BXS 為代表) 與 Arm Mali (以 Bifrost / Valhall 為代表) 在 Shader 設計維度上的關鍵對比：

設計維度	Imagination PowerVR (Rogue / Furian / BXS)	Arm Mali (Valhall / Immortalis)
基本渲染架構	嚴格 TBDR (Tile-Based Deferred Rendering)	現代 TBR (Tile-Based Rendering) 結合 IDVS
遮擋剔除機制	著色前完全硬體 HSR (Image Synthesis Processor)	Early-Z + 光柵化後 FPK (Forward Pixel Kill) 佇列
著色叢集命名	USC (Unified Shading Cluster)	Shader Core (SC) / Execution Engine (EE)
執行緒調度模型	4-lane Vertex Issue Group / 2x2 Fragment Quad 槽位	16-thread 標量 Warp (SIMT 執行緒束)
向量/標量管線	標量/向量混合、支援可配置 16/32-wide 向量運算	純標量 (Scalar) SIMT，每 Warp 16 執行緒鎖步執行
指令集架構	PCO (PowerVR Compiler Object) 雙發射+重複計數	Valhall 64-bit 標量指令，顯式分數板 Slot 遮罩
暫存器檔案	TEMP (私有), SHARED (跨緒常數/描述符), COEFF (內插)	動態 GPR (32 或 64 個 32-bit 暫存器) + Uniform 暫存器
Varying 內插	PDS 載入係數庫，USC 專用 FITRP 透視校正迭代器	Varying / SFU 專用管線指令 (LD_VAR) 搭配分數板
非同步延遲隱藏	資料請求計數器 (DRC0/1) + WDF 柵欄 + Continuation	顯式分數板槽位 (Slots 0..3) + 非阻塞排程

記憶體解耦方式	TPU 經非同步 FIFO 與 USC 解耦，由 DRC0/WDF 同步	Texture / LS 單元解耦，由 Scoreboard 遮罩解鎖
---------	--------------------------------------	-------------------------------------

1.1 幾何、平鋪與光柵化管線整合

PowerVR 流程：

1. VDM (Vertex Data Master) 接收繪圖指令並驅動 VertexFetch，透過 VertexPdsEngine 產生頂點工作。
2. 頂點資料進入 USC 執行頂點著色，輸出轉換後的座標至 ClipCull。
3. 圖元由 Tiler 依照螢幕切分之圖塊（如 16×16 或 32×32 像素 Tile）進行 Binning，將圖元幾何與狀態寫入系統記憶體中的 Parameter Buffer (PB)。
4. 在圖塊渲染階段，ISP (Image Synthesis Processor) 從 PB 讀取幾何，執行深度平面測試與覆蓋運算，徹底消除被遮蔽的幾何片元。
5. 通過 HSR 檢驗的可見圖元才被送入 UscSlot 進行 2×2 Fragment Quad 仲裁，交由 UscCluster 執行片段著色。

Arm Mali 流程：

1. IDVS (Index-Driven Vertex Shading) 將頂點著色分為兩階段：僅計算位置座標 (Position-only Shading) 用於圖元裁剪與圖塊分組，延遲一般 Varying 屬性的著色直到確認圖元可見。
2. 圖塊幾何確定後，光柵化器生成像素片元。
3. 片元首先通過 Early-Z 測試；若為不透明圖元，片元進入 FPK (Forward Pixel Kill) 緩衝區。在片元等待進入 Shader Core 執行期間，若有更新且深度更淺的片元覆蓋相同位置，舊片元在尚未執行 Shader 前便會在佇列中被直接殺死 (Killed)。

1.2 著色運算核心與執行緒拓撲

PowerVR USC 架構：

- USC 是一個高度多執行緒的執行叢集。在 Rogue 架構中，每個 USC 包含多組算術管線 (ALU Pipeline)，可並行處理頂點、片段或運算任務。
- 排程粒度：
- 頂點階段：以 4-lane Vertex Issue Group 為仲裁單元。
- 片段階段：以 2×2 Fragment Quad (共 4 像素) 為核心幾何單元，確保 **dFdx** 與 **dFdy** 導數運算的原生硬體支援。
- 叢集層級：由多個 Execution Slot 組成，支援 16-wide 或 32-wide 的實體 ALU 資料路徑。
- 雙發射架構：主 ALU 負責 32-bit 浮點 FMA/ADD/MUL 及整數運算；次 ALU 負責資料搬移 (**MBYP**)、位元操作、分支與型別轉換。

Arm Mali Valhall 架構：

- 徹底捨棄 Bifrost 的 Clause 與複雜 tuple 打包，回歸純標量 16-wide Warp (每個 Warp 包含 16 個獨立執行緒)。
- 每個 Execution Engine (EE) 配備 16 個 SIMD 運算單元，在單一週期內以標量方式同時執行一個 Warp 的 16 個執行緒。
- 雙發射能力 (Dual-Issue)：Valhall 在硬體層面支援單一 Warp 內受限的雙指令發射，例如一個 FMA 指令搭配一個 ADD/SFU/分支指令，大幅提昇指令級並行度 (ILP)。

1.3 暫存器檔案組織與分配策略

- PowerVR USC 暫存器分層：
 1. **TEMP** (Temporary Registers, **r0..r63**): 每個 Lane 私有的 32-bit 一般目的暫存器，儲存中間運算變數。
 2. **SHARED** (Shared Registers, **sh0..sh127**): 叢集內所有執行緒唯讀共享的暫存器，專門存放 Push Constants、Uniform Buffer 數值、紋理/取樣器描述符 (Descriptor Prefix)。
 3. **COEFF** (Coefficient Banks): 專屬唯讀平面係數快取，由 PDS 自 Parameter Buffer 載入，專子供 Varying 內插單元使用。
 4. **INTERNAL**: 特殊硬體暫存器，儲存執行緒 ID、重心座標、光柵狀態等。
- Arm Mali Valhall 暫存器分配：
 - 採用動態暫存器分配架構：編譯器可指定每個執行緒分配 32 個或 64 個 32-bit GPR。
 - 32 GPR 模式：核心可容納最大數量的活躍 Warp (例如每核心 64 個 Warps)，實現最佳的執行緒級並行度 (TLP) 以掩蓋記憶體延遲。
 - 64 GPR 模式：為複雜 Shader 提供更多暫存器，避免 Register Spill (溢出至記憶體)，但活躍 Warp 數量減半 (降至 32 個 Warps)。

- Uniform 暫存器：Valhall 具備獨立的 Uniform 儲存空間，避免常數讀取佔用主暫存器檔案的讀寫埠。

1.4 Varying 屬性內插機制

- ****PowerVR 的 FITRP 專用硬體****：
 - 片段著色開始前，PDS (Pixel Data Sequencer) 根據幾何面方程式自 Parameter Buffer 中取出三角形頂點的 Varying 屬性，預先填入 USC 的 Coefficient Banks。
 - USC 內建硬體內插器，執行指令 **FITRP.PIXEL** (Fragment Iteration with Perspective)。該指令以硬體單元直接讀取係數與當前 Quad 的重心座標，計算透視校正內插（包含 **1/W** 乘法）。
 - 內插過程完全非同步：發出 **FITRP** 後，硬體將請求計數器 **DRC0** 加 1；當後續指令需要該屬性值時，透由 **WDF** (Wait for Data Fence) 等待資料就緒，期間 ALU 可繼續執行其他無關指令。
- Arm Mali 的 Varying Pipeline：
 - Mali 透過專屬的 Varying 管線或特殊功能單元 (SFU) 執行 **LD_VAR** 指令。
 - 編譯器發射 **LD_VAR_FLAT** 或 **LD_VAR_PERSPECTIVE**，指令將指定一個硬體分數板 Slot，當內插計算完成後分數板自動清除，恢復相依運算。

2. 硬體分數板 (Hardware Scoreboard) 實作細節深層探討

在現代高效能平行處理器中，硬體分數板 (Hardware Scoreboard) 是防止資料相依性冒險 (RAW - Read After Write, WAR - Write After Read, WAW - Write After Write) 並隱藏非同步長延遲管線（如紋理取樣、全域記憶體讀寫、Varying 內插）的核心微架構實作。

硬體分數板 (Scoreboard) 與非同步延遲隱藏機制對比

↓ [Arm Valhall 16-wide Warp 分數板架構] 每 Warp 擁有 Slot 0..3 獨立依賴槽位 指令發射管線 (Issue Slot)

(標記 Slot 1 為 BUSY)

```
-> 直接連續發射至 ALU 執行
-> 檢測 Slot 1 是否仍為 BUSY?
|
+-- YES: Warp 進入 BLOCKED 狀態
| 調度器切換執行其他就緒 Warp
|
+-- NO (資料已返回，Slot 1 歸零):
-> 解鎖並發射相依指令
```

2.1 Arm Mali Valhall 的 16-wide Warp 硬體分數板

Valhall 架構的關鍵設計目標在於消除 Bifrost 子句暫存器溢出的限制，同時保持對記憶體非同步延遲的高度隱藏能力。

1. 分數板槽位架構 (Scoreboard Slots)

- 每個硬體 Warp 上下文 (Context) 配備了一組專屬的依賴槽位暫存器 (Dependency Slots)，典型配置為 4 個槽位 (Slot 0、Slot 1、Slot 2、Slot 3)。
- 每個槽位本質上是一個單一或微型計數的正反器 (Flip-Flop)，表示當前是否有發往特定功能單元的長延遲交易尚未結清。

2. 指令編碼與解碼語意

Valhall 的 64-bit 指令字元中包含了專屬的分數板控制欄位：

- **set_slot** (2 bits): 指示本條指令完成後將通知哪一個 Slot。當發射長延遲指令（例如 **TEX** 紋理取樣、**LD_VAR** Varying 讀取、**LOAD_GLOBAL** 記憶體讀取）時，指定將 Slot **K** 設為有效 (Set Slot **K** Pending)。
- **wait_slots** (4-bit bitmask): 指示當前指令在發射之前，必須等待哪些 Slot 清空。若 **wait_slots** = **0b0101**，表示指令必須等到 Slot 0 與 Slot 2 均完成回寫後才能發射。

3. Warp 調度器與非阻塞發射 (Non-blocking Issue)

- 就緒仲裁 (Ready Arbitration)：調度器在每個時脈週期評估所有活躍的 16-wide Warps。一個 Warp 被判定為「就緒 (Ready)」，當且僅當：

WarpReady = (Instruction.wait_slots & Warp.PendingSlots) == 0

• 延遲隱藏實例：

; Valhall 組語示例：延遲隱藏
TEX.2D r0..r3, r4.xy, slot(0) ; 發出取樣，設置 Slot 0 pending
FADD r10, r11, r12 ; 獨立運算，wait_slots = 0 (不阻塞，立即發射)
FMUL r13, r14, r15 ; 獨立運算，wait_slots = 0 (不阻塞，立即發射)
FADD r20, r0, r10, wait(0) ; 需要紋理結果 r0，wait_slots = 0b0001
; 若 Slot 0 仍 pending，Warp 停頓並讓出發射槽

2.2 PowerVR USC 的 DRC 計數器與 Continuation 機制

PowerVR 在硬體複雜度與面積控制上展現了極致的工程取捨。它不為每組通用暫存器配置複雜的槽位矩陣，而是採用了資料請求計數器 (Data Request Counter, DRC) 搭配 接續 (Continuation) 機制。

1. DRC 架構與分類

每個 USC 執行緒槽位具備兩個專屬的硬體向上/向下計數器 (Up/Down Counter)：

- **DRC0**：主要追蹤 TPU (紋理取樣單元) 請求與 FITRP (Varying 平面內插單元) 請求。
- **DRC1**：主要追蹤 全域/區域記憶體 (Memory Load/Store) 與參數緩衝區事務。

2. DRC 運作流程

1. 發起請求 (Dispatch)：當 USC 執行 **SMP** (Texture Sample) 或 **FITRP** 時，硬體微碼自動向外部非同步單元送出事務封包，同時將 **DRC0** 計數值加 1：

DRC0 ← **DRC0** + 1

2. 流水線自由前進：後續只要是不依賴該返回資料的指令 (例如座標變換、色彩係數混合、常數加載)，USC 均照常按週期發射執行，完全不浪費流水線週期。

3. ****資料柵欄 (WDF)****：當程式抵達必須消費返回結果的關鍵節點時，編譯器會插入一條 **WDF** (Wait for Data Fence) 指令：

WDF.drc0, 0 ; 宣告：等待 DRC0 計數器遞減至小於或等於 0

3. 槽位掛起與接續機制 (Continuation Mechanism)

在真實硬體與 SystemC 模擬中，當執行緒在 **WDF** 發現 **DRC0 > 0** 時：

- 狀態儲存：USC 硬體不會凍結整個核心的時脈，而是將該執行緒槽位標記為 **WAITING_DRC0**，並凍結其程式計數器 (PC)。
- 微架構切換：USC 槽位仲裁器 (**UscSlot**) 立即將 ALU 的發射週期讓給其他狀態為 **READY** 的 2x2 Fragment Quad 或 4-lane Vertex Group。
- 資料返還與喚醒：當 TPU 或 MemoryPool 經由非同步 FIFO 將 4 個 DWORD 的色彩/內插資料回傳至 USC 時，硬體將回傳資料填入指定的暫存器目標 (如 **TEMP0..3**)，並將 **DRC0** 減 1：

DRC0 ← **DRC0** - 1

- 當 **DRC0 == 0** 時，該槽位被標記為就緒，於下一個仲裁週期恢復 (Resume) 在 **WDF** 之後的指令執行。

2.3 兩種硬體分數板實作的深度技術對比

比較特性	Arm Mali Valhall 分數板 (Slots)	PowerVR USC 分數板 (DRC + WDF)
追蹤粒度	顯式指定之槽位編號 (Slot 0..3)	事件計數器 (DRC0 / DRC1)
硬體狀態開銷	每個 Warp 需 4~8 組狀態正反器與比較電路	每個 Slot 僅需 2 個小位元計數器 (如 3-bit counter)
矽晶面積與漏電	較高 (需支援多槽位遮罩解碼與比對)	極低 (簡單加減計數器與比較歸零邏輯)
亂序返回支援	支援多個不同 Slot 之間的亂序完成	相同 DRC 管道內預設為保序 (In-Order) 返回
編譯器複雜度	需透過圖著色法最佳化 Slot 分配，避免虛假依賴	簡單直觀：發出請求後插入無關指令，最後插入 WDF

暫存器衝突檢測	整合暫存器遮罩比對，預防 RAW/WAR 衝突	短延遲 ALU 依賴旁路網路；長延遲完全靠 WDF 隔絕
---------	-------------------------	------------------------------

3. PowerVR USC 微架構之 SystemC 模擬實現

在本專案的 `src/systemc/` 目錄中，完整實現了包含 36 個 SystemC 模組的週期精確/事件驅動型 PowerVR GPU 架構。特別是 `src/systemc/shader/` 子系統，忠實模擬了 PCO 指令解碼、USC 槽位仲裁、PCO ISS 執行、DRC0 計數器維護與 Continuation 非同步延遲隱藏。

3.1 著色核心模組架構與職責劃分

SystemC Shader Subsystem Architecture (`src/systemc/shader/`)

↓ 來自上游管線 (VertexPds / ParameterBuffer) `sc_fifo_in<PipelineTxn>`

① `PcoDecoder` | 將 Mesa 產出之 PCO 機器碼 (PCO Binary) (`pco_decoder.cpp`) | 解碼為 `PcoInstruction[]` 結構體與指令組

↓ `sc_fifo<PipelineTxn>`

② `UscSlot` | 仲裁 4-lane Vertex Group 與 2x2 Fragment Quad (`usc_slot.cpp`) | 維護執行槽位合法性與狀態轉移

↓ `sc_fifo<PipelineTxn>`

③ `UscCluster` | USC 叢集執行單元：
(`usc_cluster.cpp`) | - 綁定 `MemoryPool` 與 `PipelineState`
| - 串接非同步 TPU (`texture_request/response` FIFO)
| - 處理 SMP 掛起與 WDF 接續

↓ (內部調用)

④ `Pcolss (Instruction Set Simulator, pco_iss.cpp)`
- 模擬暫存器檔案: VTXIN/OUT, PIXOUT, TEMP, SHARED, COEFF
- 模擬 ALU 指令: FADD, FMUL, FMAD, MBYP 等純整數演算法
- 模擬 FITRP: 讀取係數庫與重心座標進行透視校正內插
- 模擬 WDF: 依賴計數驗證，若未就緒則引發懸掛等待

3.2 關鍵微架構資料結構與 SystemC 通訊協定

在 `src/systemc/common/pipeline_state.h` 與 `src/systemc/shader/pco_iss.h` 中，定義了極其精確的硬體執行狀態：

```
// PcoInstruction: 語意微指令表示
struct PcoInstruction {
    std::uint32_t binary_offset; // 二進位偏移量
    std::uint16_t group_index;   // 所屬指令組編號
    PcoOpcode opcode;           // 微操作碼 (kFloatAdd, kTextureSample, kWaitDataFence...)
    PcoWriteTarget target;       // 寫入目標 (kTemporary, kPixelOutput, kVertexOutput)
    std::uint16_t output_index;  // 目標暫存器索引
    std::uint16_t component_count; // 向量維度 (1..4)
    std::uint16_t repeat_count;  // PCO 特有之硬體重複發射次數
    PcoRegisterRef source0;      // 來源暫存器 0
    PcoRegisterRef source1;      // 來源暫存器 1
    std::uint8_t data_request;    // 關聯之非同步資料請求通道 (DRC0 / DRC1)
};

// PcolssContinuation: 執行緒接續上下文
struct PcolssContinuation {
    std::uint32_t valid;         // 接續是否有效
    std::uint32_t program_counter; // 暫停時之 PC
    std::uint32_t shared_count;  // 有效常數/描述符數量
    std::uint32_t emitted;       // 頂點輸出發射標記
    std::uint32_t ended_task;    // 任務終止標記
};
```

};

3.3 USC DRC 分數板與 Continuation 實作原始碼剖析

在 `src/systemc/shader/pco_iss.cpp` 中，對 **WDF**、**SMP** 與 **FITRP** 的計數器依賴處理展示了 PowerVR 硬體分數板的真實運作：

1. 非同步發射與掛起 (Suspend)：

當遇到紋理取樣指令 **kTextureSample**，若當前尚無非同步回應，ISS 會打包紋理取樣請求，記錄當前狀態至 Continuation，並暫停當前 Lane 的執行：

```
// 於 pco_iss.cpp 中：遇到紋理取樣時進行非同步派發與暫停
if (instruction.opcode == PcoOpcode::kTextureSample) {
    result.texture_request.valid = 1;
    result.texture_request.shader_lane_index = lane_index;
    result.texture_request.quad_id = quad_id;
    // 儲存接續狀態，讓出執行權
    result.continuation.valid = 1;
    result.continuation.program_counter = pc;
    result.suspended = 1;
    return result; // 退出 ISS，由 UscCluster 切換其他槽位
}
```

2. 分數板等待驗證 (kwaitDataFence)：

當執行抵達 **WDF** 時，ISS 檢驗 **drc0_pending** 標記。若先前有請求發出且資料未抵達，則嚴禁前進；當資料回填後，清空 **drc0_pending** 標記並寫入暫存器：

```
// 於 pco_iss.cpp 中：WDF 指令確保暫存器寫入安全
if (instruction.opcode == PcoOpcode::kWaitDataFence) {
    if (!drc0_pending) {
        ExecuteError("WDF did not match one pending drc0 request");
    }
    // 將非同步傳回之數值安全寫入 Temporary 暫存器檔案
    for (std::size_t c = 0; c < pending_component_count; ++c) {
        temporaries[pending_output_index + c] = pending_data[c];
        temporary_written_mask |= (UINT64_C(1) << (pending_output_index + c));
    }
    drc0_pending = false; // DRC0 柵欄解除
    ++pc;
    continue;
}
```

3. 嚴格防護 RAW 資料冒險：

若 Shader 程式試圖在 **WDF** 完成前讀取該暫存器，ISS 會立即觸發硬體異常：

```
if (!(temporary_written_mask & (UINT64_C(1) << reg_idx))) {
    ExecuteError("fragment TEMP read occurred before WDF completion");
}
```

4. 已公開 Shader 效能計數器 (Performance Counters) 完整矩陣

效能計數器 (Performance Counters) 是架構分析、驅動調優與遊戲瓶頸診斷的基石。以下系統性彙整了 Imagination PowerVR (PVRTune、PVRCarbon、Mesa JSONL 協定) 與 Arm Mali (Streamline、Mali Graphics Debugger、Panfrost) 所公開的所有核心 Shader 效能計數器。

4.1 PowerVR 效能計數器體系 (PVRTune & Mesa PvrGPU)

在 PowerVR 架構中，計數器由 GPU 內部微控制器 (如 META 或 MIPS 韌體核心) 定期自硬體計數單元取樣，或透過 Mesa 驅動與 SystemC 模型中的 `drawlist_stats[]` JSONL 串流回傳。

計數器名稱 (Counter Name)	統計層級	資料來源/協定	物理意義與分析指標
----------------------	------	---------	-----------

pco_instructions	靜態程式	Mesa / PvrGPU	Shader 靜態解碼之語意指令總數（不計 repeat 展開）
vs_alu_instructions	動態執行	HW / SystemC	頂點著色階段動態執行之 ALU 指令總數（含 repeat 與 lanes 乘積）
vs_tex_instructions	動態執行	HW / SystemC	頂點著色階段執行之 SMP 紋理取樣次數
vs_memory_instructions	動態執行	HW / SystemC	頂點輸出與緩衝區存取次數（如 UVSW.write）
fs_alu_instructions	動態執行	HW / SystemC	片段著色階段動態執行之 ALU 運算總數（包含色彩混合與浮點計算）
fs_tex_instructions	動態執行	HW / SystemC	片段著色階段動態執行之紋理取樣次數
fs_memory_instructions	動態執行	HW / SystemC	片段著色階段常數與緩衝區讀寫次數
ps_invocations	光柵統計	PVRTune / HW	通過 HSR 檢驗並真正進入 USC 執行的片段呼叫總數
vs_invocations	幾何統計	PVRTune / HW	進入 USC 執行的頂點著色呼叫總數
covered_pixels	光柵統計	HW / SystemC	光柵化後三角形覆蓋的總像素數
fragment_candidates	光柵統計	HW / SystemC	進入 ISP 深度排序與剔除前的候選片元數
hsr_rejected_fragments	HSR 效率	PVRTune / HW	極關鍵指標：被 ISP HSR 成功剔除、未消耗 USC 運算的片元數
drc_stall_cycles	調度停頓	PVRTune / HW	USC 執行緒因等待 WDF（DRC0/1 > 0）而處於掛起狀態的時脈週期數
fifo_stall_events	匯流排停頓	SystemC / HW	USC 與 TPU / MemoryPool 之間 FIFO 佇列滿載引發的反壓停頓次數
virtual_gpu_cycles	總體計時	SystemC / HW	模擬 GPU 運行的虛擬總時脈週期
tpu_requests / responses	紋理管線	PVRTune / HW	發往 TPU 的取樣請求封包數與傳回的像素色彩封包數
pool_bytes_in_flight	記憶體頻寬	SystemC / HW	目前正在記憶體交換網路上傳輸或佇列中的資料量 (Bytes)
pbe_fragment_writes	像素後端	HW / SystemC	著色完成後寫入 PBE (Pixel Back End) 圖塊緩衝區的片元數

4.2 Arm Mali 效能計數器體系 (Arm Streamline & Panfrost)

Arm Mali 在每個 Shader Core (SC) 與頂層記憶體單元均配置了 64 個硬體效能計數器暫存器，透過 Mali 核心驅動（kbase）以每秒數十至數百次的頻率進行輪詢。

計數器名稱 (Mali Counter)	統計模組	物理意義與分析指標
GPU_ACTIVE	全域核心	GPU 至少有一個硬體管線單元在工作時的時脈週期數
WARP_CYCLES	Execution Engine	核心內所有 Warps 執行的累積週期數（評估計算負載總量）
WARPS_ISSUED	Warp Scheduler	Warp 調度器成功派發指令的總次數
WARP_DIVERGENCE	Warp Control	關鍵分支指標：因 16-wide 執行緒內部條件分支不一致導致分歧執行的週期數

SCOREBOARD_STALL_CYCLES	Scoreboard	核心依賴指標：Warp 就緒但因等待 Scoreboard Slot 釋放而停頓的週期數
EE_STARVATION	Execution Engine	執行引擎處於飢餓狀態（所有 Warps 均未就緒，無法發射指令）的週期數
ALU_ACTIVE_CYCLES	ALU Pipeline	主運算單元（FMA/ADD/SUB）處於活躍計算狀態的週期數
FMA_PIPE_ACTIVE	ALU Pipeline	專屬 FMA 乘法管線工作週期數（評估 Shader 浮點運算強度）
SFU_PIPE_ACTIVE	SFU Pipeline	特殊功能單元（ sin , cos , √x , log ）計算週期數
LS_ISSUES	Load/Store Unit	記憶體載入與儲存管線派發的指令數
TEX_ISSUES	Texture Unit	紋理映射單元（TMU）接收並處理的取樣指令總數
TEX_FILTER_CYCLES	Texture Unit	紋理雙線性（Bilinear）或三線性（Trilinear）過濾單元運作週期數
TEX_CACHE_MISS	Texture Cache	紋理 L1/L2 快取未命中次數（代表紋理頻寬壓力與局部性差）
VARYING_ACTIVE_CYCLES	Varying Pipe	執行 Varying 內插運算的週期數
FPK_KILLED_QUADS	光柵與 FPK	Mali 獨家指標：進入 FPK 佇列後成功被覆蓋圖元殺死的 2x2 Quad 數量
EARLY_Z_KILLED_QUADS	光柵單元	在光柵化早期被 Early-Z 測試直接剔除的 Quad 數量

4.3 效能瓶頸診斷交叉分析矩陣 (Bottleneck Diagnosis)

透過上述兩大架構的效能計數器，工程師可精確定位以下四種典型 Shader 瓶頸：

GPU Shader 瓶頸特徵與診斷矩陣

5. 結論與未來微架構演進展望

1. 架構哲學的融合：
- PowerVR 透過 TBDR 與 HSR 建立了極高的能效壁壘，其 DRC 與 **WDF** 分數板以極小矽晶面積實現了高效延遲隱藏。
 - Arm Mali 自 Valhall 開始導入 16-wide 純標量 SIMT 與顯式分數板 Slot 架構，並在 Immortalis 與第 5 代架構中進一步導入了光線追蹤硬體單元（RTU）與延遲頂點著色（DVS），兼具編譯器友好性與極致的計算吞吐。

2. 現代硬體分數板的發展：

現代行動 GPU 的分數板正朝向「軟硬體協同設計」邁進：編譯器在靜態編譯期完成暫存器生命週期分析與槽位圖著色，硬體僅需維護輕量化的位元遮罩與非同步計數器，在大幅降低指令解碼功耗的同時，提供高達數百週期的記憶體延遲容忍度。

3. SystemC 建模的實證價值：

本專案所建構之 36 個 SystemC 模組證明：即使面對高度複雜的 USC 著色核心、非同步 TPU 連接埠與多層快取一致性，透過 Transaction-Level 與 Event-Driven 建模，依然能夠在軟體模擬環境中重現真實晶片的指令執行特性、延遲隱藏行為與效能計數器指標，為次世代行動 GPU 的演算法探索與架構驗證提供了堅實的基礎。